

UNIVERSIDAD AUTÓNOMA DEL ESTADO DE HIDALGO

INSTITUTO DE CIENCIAS BÁSICAS E INGENIERÍA

LICENCIATURA EN CIENCIAS COMPUTACIONALES

Autómatas y Compiladores

PRÁCTICA 4

ANÁLISIS LÉXICO DE PALABRAS

Alumno: Pilar Bucio

Semestre: 6°

Profesor: Eduardo Cornejo Velázquez

1. Introducción

El **analizador léxico** es la primera fase de un compilador. Su principal función consiste en leer los caracteres de entrada y elaborar como salida una secuencia de componentes léxicos que utiliza el analizador sintáctico para hacer su análisis.

El **análisis léxico** también es llamado exploración o *scanner*. Su función es leer caracteres uno a uno desde la entrada e ir formando grupos de caracteres con alguna relación entre sí, llamados *tokens*, los que serán la entrada para la siguiente etapa del compilador.

El analizador léxico:

- Es la primera fase del programa traductor.
- Es el único que gestiona el archivo de entrada con el código fuente.
- Es el que lee los caracteres del programa fuente y construye símbolos intermedios, los cuales serán la entrada del analizador sintáctico.

El analizador léxico típicamente detecta los siguientes errores:

- Utilizar caracteres que no pertenecen al alfabeto del lenguaje.
- Cadenas que no coinciden con ninguno de los patrones de los *tokens* posibles.

2. Objetivo general

Construir programas en un lenguaje de programación de alto nivel que realicen el análisis de cadenas de símbolos que forman palabras, basado en la implementación de estructuras de datos y algoritmos de análisis, además del uso de expresiones regulares como segunda estrategia de implementación.

3. Objetivos específicos

- Identificar el proceso básico de análisis de cadenas de símbolos de un alfabeto.
- Implementar estructuras de datos y algoritmos de análisis de cadenas de símbolos en un lenguaje de programación de alto nivel.
- Implementar expresiones regulares para el análisis de cadenas de símbolos en un lenguaje de programación de alto nivel.

4. Equipo y software a utilizar

Se utilizará el software FLEX para Windows.

5. Procedimiento para el desarrollo de la práctica

Realizar los siguientes programas en FLEX:

1. Reconocer números enteros.
2. Reconocer números decimales.
3. Reconocer variables (identificadores). Inician con una letra y después pueden tener letras, números y guiones bajos.
4. Reconocer RFC.
5. Reconocer CURP.
6. Reconocer correo electrónico institucional.

Compilación y ejecución desde línea de comandos

Para la compilación desde línea de comandos utilizando FLEX se emplean las siguientes instrucciones:

```
C:\>flex [nombre-programa].l
```

```
C:\>gcc lex.yy.c -o [nombre-ejecutable]
```

Para la ejecución desde línea de comandos se utiliza la siguiente instrucción:

```
C:\>[nombre-ejecutable].exe
```

Programa 1. Reconocimiento de números enteros

```
%{
#include <stdio.h>
%}

%%

[0-9]+      { printf("Numero entero: %s\n", yytext); }

[ \t\n]+    ;

.           { printf("No es un numero entero: %s\n", yytext); }

%%

int yywrap()
{
    return 1;
}

int main()
{
    printf("Ingresa una cadena:\n");
    yylex();

    return 0;
}
```

```
---------- Lex File Compile ----------
Output completed (0 sec consumed) - Normal Termination
```

enteros.1

Figura 1: Código del programa 1: reconocimiento de números enteros.

```
C:\Users\pilar\OneDrive\Desktop\FLEX>lex.yy.c

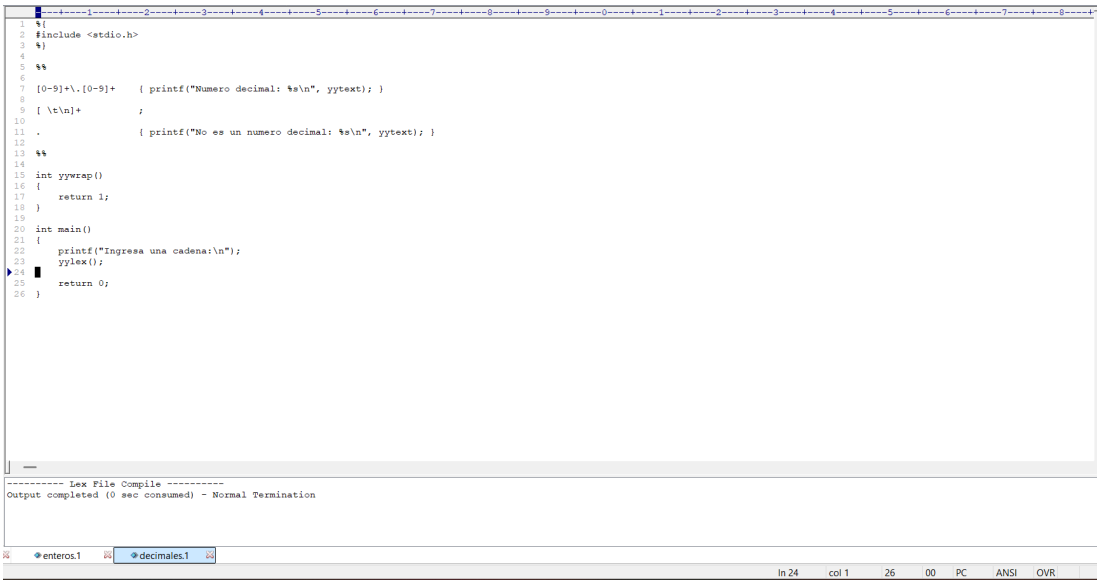
C:\Users\pilar\OneDrive\Desktop\FLEX>gcc lex.yy.c -o enteros

C:\Users\pilar\OneDrive\Desktop\FLEX>enteros.exe
Ingresa una cadena:
25 100 7
Numero entero: 25
Numero entero: 100
Numero entero: 7

enteros.exe
No es un numero entero: e
No es un numero entero: n
No es un numero entero: t
No es un numero entero: e
No es un numero entero: r
No es un numero entero: o
No es un numero entero: s
No es un numero entero: .
No es un numero entero: e
No es un numero entero: x
No es un numero entero: e
25 hola 2
Numero entero: 25
No es un numero entero: h
No es un numero entero: o
No es un numero entero: l
No es un numero entero: a
```

Figura 2: Compilación y ejecución del programa 1.

Programa 2. Reconocimiento de números decimales



```
%{
#include <stdio.h>
%}

%%

[0-9]+\.[0-9]+    { printf("Numero decimal: %s\n", yytext); }

[ \t\n]+          ;

.                 { printf("No es un numero decimal: %s\n", yytext); }

%%

int yywrap()
{
    return 1;
}

int main()
{
    printf("Ingresa una cadena:\n");
    yylex();

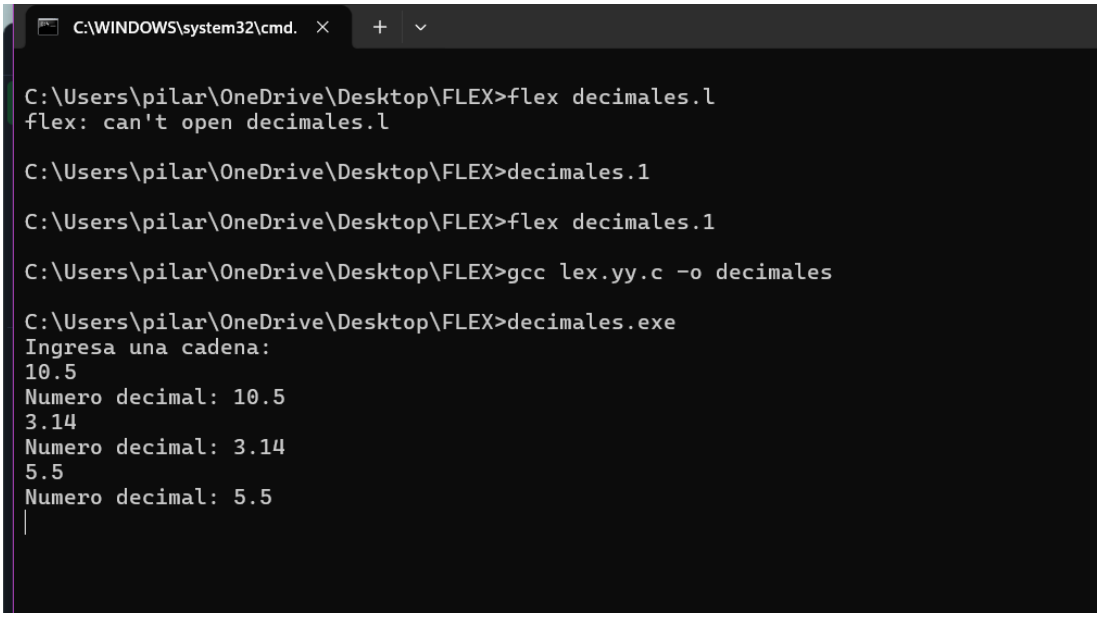
    return 0;
}
```

```
---------- Lex File Compile ----------
Output completed (0 sec consumed) - Normal Termination
```

enteros.1 decimales.1

ln 24 col 1 26 00 PC ANSI OVR

Figura 3: Código del programa 2: reconocimiento de números decimales.



```
C:\WINDOWS\system32\cmd.

C:\Users\pilar\OneDrive\Desktop\FLEX>flex decimales.l
flex: can't open decimales.l

C:\Users\pilar\OneDrive\Desktop\FLEX>decimales.1

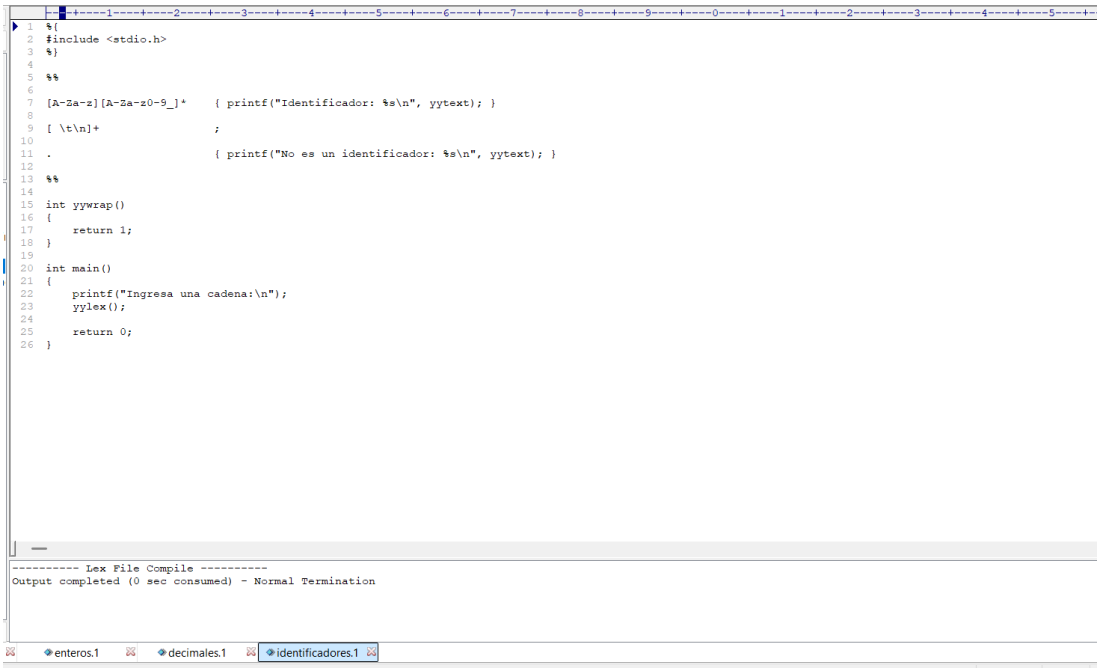
C:\Users\pilar\OneDrive\Desktop\FLEX>flex decimales.1

C:\Users\pilar\OneDrive\Desktop\FLEX>gcc lex.yy.c -o decimales

C:\Users\pilar\OneDrive\Desktop\FLEX>decimales.exe
Ingresa una cadena:
10.5
Numero decimal: 10.5
3.14
Numero decimal: 3.14
5.5
Numero decimal: 5.5
```

Figura 4: Compilación y ejecución del programa 2.

Programa 3. Reconocimiento de variables



```
%{
#include <stdio.h>
%}

%%

[A-Za-z][A-Za-z0-9_]*    { printf("Identificador: %s\n", yytext); }

[ \t\n]+                 ;

.                        { printf("No es un identificador: %s\n", yytext); }

%%

int yywrap()
{
    return 1;
}

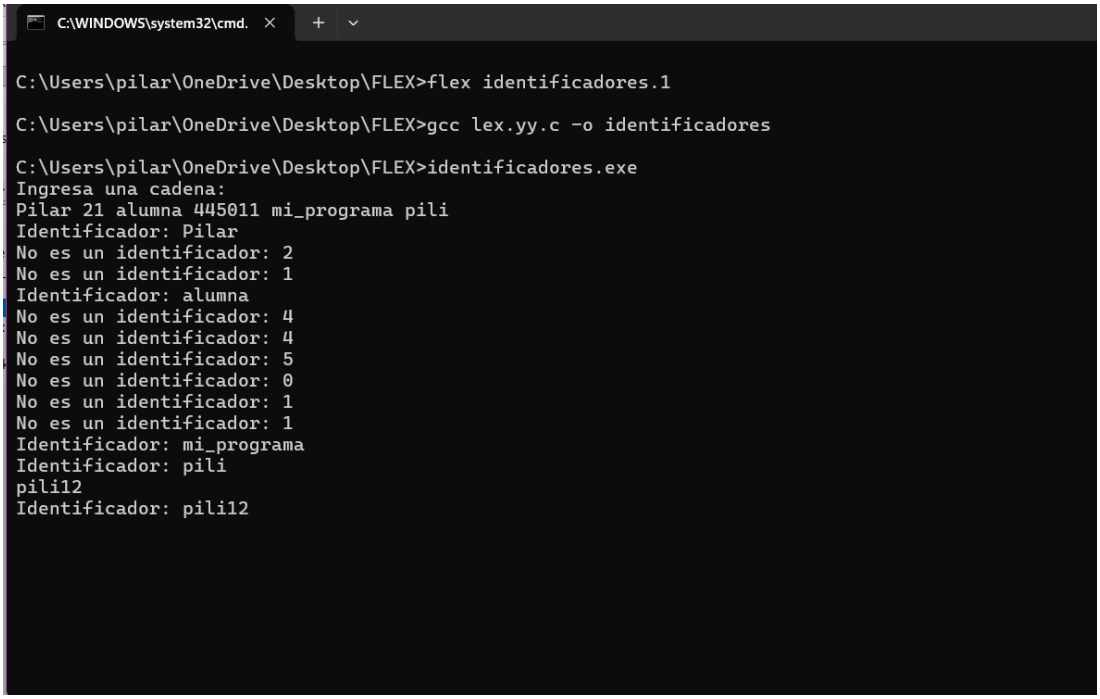
int main()
{
    printf("Ingresa una cadena:\n");
    yylex();

    return 0;
}
```

```
---------- Lex File Compile ----------
Output completed (0 sec consumed) - Normal Termination
```

enteros.1 | decimales.1 | identificadores.1

Figura 5: Código del programa 3: reconocimiento de variables o identificadores.



```
C:\WINDOWS\system32\cmd.

C:\Users\pilar\OneDrive\Desktop\FLEX>flex identificadores.1

C:\Users\pilar\OneDrive\Desktop\FLEX>gcc lex.yy.c -o identificadores

C:\Users\pilar\OneDrive\Desktop\FLEX>identificadores.exe
Ingresa una cadena:
Pilar 21 alumna 445011 mi_programa pili
Identificador: Pilar
No es un identificador: 2
No es un identificador: 1
Identificador: alumna
No es un identificador: 4
No es un identificador: 4
No es un identificador: 5
No es un identificador: 0
No es un identificador: 1
No es un identificador: 1
Identificador: mi_programa
Identificador: pili
pili12
Identificador: pili12
```

Figura 6: Compilación y ejecución del programa 3.

Programa 4. Reconocimiento de RFC

```
C:\Users\pilar\OneDrive\Desktop\FLEX>flex rfc.1

C:\Users\pilar\OneDrive\Desktop\FLEX>gcc lex.yy.c -o rfc

C:\Users\pilar\OneDrive\Desktop\FLEX>rfc.exe
Ingresa un RFC:
BUPJ010203ABC
RFC valido: BUPJ010203ABC
BUHP050207MDFCDLA123
No es un RFC valido: BUHP050207MDFCDLA123
```

Figura 7: Código del programa 4: reconocimiento de RFC.

```
%{
#include <stdio.h>
%}

%%

[A-ZÑ&]{4}[0-9]{6}[A-Z0-9]{3}    { printf("RFC valido: %s\n", yytext); }

[A-ZÑ&]{3}[0-9]{6}[A-Z0-9]{3}    { printf("RFC valido: %s\n", yytext); }

[ \t\n]+                          ;

[A-Za-z0-9]+                      { printf("No es un RFC valido: %s\n", yytext); }

.                                 { printf("No es un RFC valido: %s\n", yytext); }

%%

int yywrap()
{
    return 1;
}

int main()
{
    printf("Ingresa un RFC:\n");
    yylex();

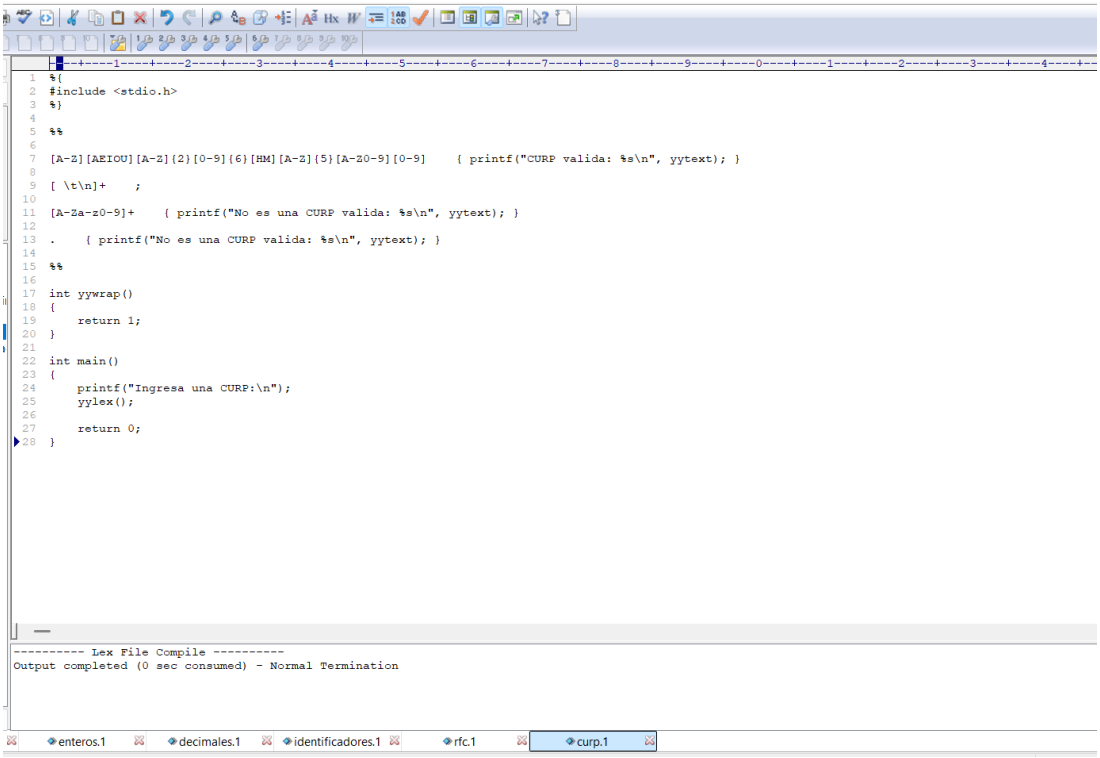
    return 0;
}
```

```
---------- Lex File Compile ----------
Output completed (0 sec consumed) - Normal Termination
```

enteros.1 | decimales.1 | identificadores.1 | rfc.1

Figura 8: Compilación y ejecución del programa 4.

Programa 5. Reconocimiento de CURP



```
%{
#include <stdio.h>
%}

%%

[A-Z][AEIOU][A-Z]{2}[0-9]{6}[HM][A-Z]{5}[A-Z0-9][0-9]    { printf("CURP valida: %s\n", yytext); }

[ \t\n]+    ;

[A-Za-z0-9]+    { printf("No es una CURP valida: %s\n", yytext); }

.    { printf("No es una CURP valida: %s\n", yytext); }

%%

int yywrap()
{
    return 1;
}

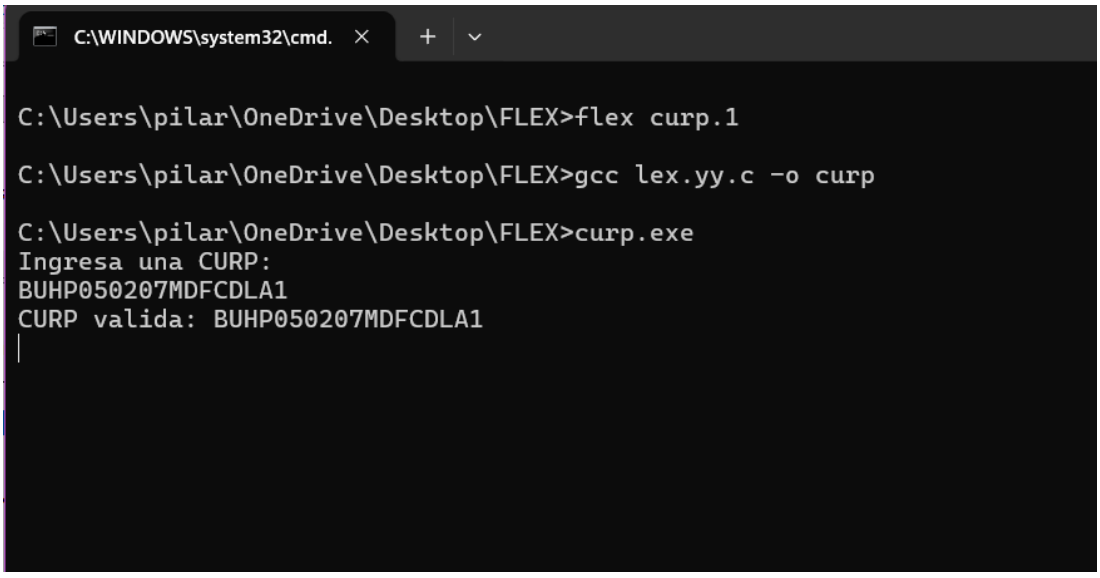
int main()
{
    printf("Ingresa una CURP:\n");
    yylex();

    return 0;
}
```

```
---------- Lex File Compile ----------
Output completed (0 sec consumed) - Normal Termination
```

enteros.1 | decimales.1 | identificadores.1 | rfc.1 | curp.1

Figura 9: Código del programa 5: reconocimiento de CURP.



```
C:\WINDOWS\system32\cmd.

C:\Users\pilar\OneDrive\Desktop\FLEX>flex curp.1

C:\Users\pilar\OneDrive\Desktop\FLEX>gcc lex.yy.c -o curp

C:\Users\pilar\OneDrive\Desktop\FLEX>curp.exe
Ingresa una CURP:
BUHP050207MDFCDLA1
CURP valida: BUHP050207MDFCDLA1
```

Figura 10: Compilación y ejecución del programa 5.

Programa 6. Reconocimiento de correo electrónico institucional

```
%{
#include <stdio.h>
%}

%%

[A-Za-z0-9. %+-]+@uaeh\.edu\.mx    { printf("Email institucional valido: %s\n", yytext); }

[ \t\n]+    ;

[A-Za-z0-9. %+-]+@[A-Za-z0-9.-]+    { printf("No es un email institucional: %s\n", yytext); }

[A-Za-z0-9._%+-]+    { printf("No es un email valido: %s\n", yytext); }

.    { printf("Caracter no valido: %s\n", yytext); }

%%

int yywrap()
{
    return 1;
}

int main()
{
    printf("Ingresa un email:\n");
    yylex();

    return 0;
}
```

```
---------- Lex File Compile ----------
Output completed (0 sec consumed) - Normal Termination
```

enteros.1 | decimales.1 | identificadores.1 | rfc.1 | curp.1 | email.1

Figura 11: Código del programa 6: reconocimiento de correo electrónico institucional.

```
C:\WINDOWS\system32\cmd.

C:\Users\pilar\OneDrive\Desktop\FLEX>flex email.1

C:\Users\pilar\OneDrive\Desktop\FLEX>gcc lex.yy.c -o email

C:\Users\pilar\OneDrive\Desktop\FLEX>email.1

C:\Users\pilar\OneDrive\Desktop\FLEX>email.exe
Ingresa un email:
pilar@uaeh.edu.mx
Email institucional valido: pilar@uaeh.edu.mx
```

Figura 12: Compilación y ejecución del programa 6.

7. Cuestionario

1. **¿Cuál es la utilidad de las expresiones regulares en la identificación de palabras de un lenguaje?**

   Sirven para definir patrones y poder identificar si una palabra o cadena cumple con ciertas características.

2. **¿Cuáles elementos del lenguaje de programación utilizaste para la implementación de las estructuras de datos? ¿Cómo las utilizaste?**

   Utilicé principalmente expresiones regulares, condiciones y funciones. Las expresiones regulares me permitieron definir los patrones que debía reconocer cada programa.

3. **¿Cuál es el proceso que seguiste para analizar las cadenas de símbolos?**

   Primero definí el patrón que quería reconocer, después lo implementé en FLEX y finalmente compilé y probé el programa con diferentes cadenas.

4. **¿Qué elementos del lenguaje de programación te permitieron implementar expresiones regulares?, ¿qué condiciones o recomendaciones se deben seguir al escribir las expresiones regulares?**

   Utilicé caracteres, rangos, operadores y cuantificadores que proporciona FLEX. Se debe escribir correctamente el patrón y considerar las características que debe cumplir la cadena para evitar reconocer datos incorrectos.

8. Bibliografía

- Giró, J., Vázquez, J., Meloni, B., & Constable, L. (2015). *Lenguajes formales y teoría de autómatas*. Editorial Alfaomega. Argentina.
- Ruiz Catalán, J. (2010). *Compiladores: teoría e implementación*. Editorial Alfaomega. México.
- Brookshear, J. G. (1995). *Teoría de la Computación. Lenguajes formales, autómatas y complejidad*. Editorial Addison-Wesley Iberoamericana. Primera edición. U.S.A.